HashDit

Smart Contract Security

ListaDao CollateralYieldVault & RewardHarvester

Security Audit Report

ListaDao

08 Jun 2026 – 15 June 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	08 Jun 2026 — 15 June 2026
Overall Risk	**High**
Project Name	ListaDao
Github	https://github.com/lista-dao/moolah/pull/188
Commit	feature/collateral-yield-vault

Smart Contract List

No.	Contract Name	Verdict	Details
1	CollateralYieldVault.sol	High	[H01] [L01] [I01] [I02] [I03] [I04] [I05] [I06] [I09]
2	RewardHarvester.sol	High	[H01] [I07] [I08]
3	deploy_collateral_yield_vault.sol	Green	-
4	ICollateralYieldVault.sol	Green	-
5	IProvider.sol	Green	-
6	IStakeManager.sol	Green	-

Findings

[H01] Yield accrued by increaseVaultAssets can be captured by short duration deposits around harvest	
Contract	CollateralYieldVault.sol, RewardHarvester.sol
Severity	High
Description	**Description** The increaseVaultAssets function raises totalAssets() without minting any new shares, so the resulting increase in value per share is awarded to whoever holds shares at the exact moment it executes. Because the vault enforces no minimum holding period, charges no deposit or withdraw fee and applies no time weighting to yield,, an actor can deposit a large amount immediately before the public harvest transaction and redeem immediately after. This lets the actor capture yield that was intended for genuine long term holders while holding for nearly zero duration, diluting the rewards that honest depositors should receive. JavaScript `function increaseVaultAssets() external payable override onlyRole(BOT) nonReentrant whenNotPaused { if (msg.value == 0) revert ZeroAmount(); uint256 balBefore = IERC20(SLIS_BNB).balanceOf(address(this)); STAKE_MANAGER.deposit{ value: msg.value }(); uint256 slisDelta = IERC20(SLIS_BNB).balanceOf(address(this)) - balBefore; PROVIDER.supplyCollateral(marketParams, slisDelta, address(this), ""); // Inject first, accrue after: the increment is booked as interest and charged `fee` (0 at launch). _updateLastTotalAssets(_accrueFee()); emit IncreaseVaultAssets(_msgSender(), msg.value, slisDelta); }` **Impact** Such front-running can dilute honest holders' share of each harvest by the attacker's deposit fraction for one block. This is profitable when a harvest is large relative to TVL.

Recommendation	Consider linearly vesting the reward harvesting, where the slisDelta can be assign as a reward that unlocks linearly over a cycle (e.g. the harvest interval). totalAssets() returns position – lockedProfit(t). A JIT depositor entering at block N and exiting at N+1 captures only one block's worth of drip exactly
Status	Open. It is presumed that a fix would be considered in the future, considering it is listed as an TODO within this commit, which was removed 4d33a9a8afd5e4ce9e471693e93e451483361588.

[L01] sweep is permanently unreachable due to the dead seed deposit	
Contract	CollateralYieldVault.sol
Severity	Low

Description	**Description**
	The RewardHarvester.sweep() function can only execute its body when totalSupply() == 0, because it reverts with SharesOutstanding() whenever totalSupply() != 0. The deployment script in deploy_collateral_yield_vault.sol does the opposite. It mints a permanent seed to the burn address using depositBNB to DEAD and asserts that totalSupply() == seedShares && seedShares > 0. This is also indicated in the natspec of the sweep function, which is performed to prevent inflation attacks on ERC4626 vaults
	
	Note that CollateralYieldVault does not override decimal offsets for added protection, since the assumption here is that slisBNB is an 18 decimal token and thus makes inflation attack unprofitable
	
	JavaScript
	`/// @notice Recover residual collateral left in the position once every share has been redeemed`
	`///  (last-redeemer rounding dust), and reset the vault to a clean empty state.`
	`/// @dev Gated on totalSupply() == 0: with no shares outstanding nothing backs user value, so this can never`
	`///  touch share-backed funds. A deploy-time dead seed deposit keeps supply > 0 and prevents this state.`
	`function sweep(address to) external onlyRole(MANAGER) nonReentrant returns (uint256 amount) {`
	`if (totalSupply() != 0) revert SharesOutstanding();`
	`if (to == address(0)) revert ErrorsLib.ZeroAddress();`
	`amount = totalAssets();`
	`if (amount > 0) {`
	`PROVIDER.withdrawCollateral(marketParams, amount, address(this), to);`
	`_updateLastTotalAssets(0);`
	`}`

```
    emit Swept(to, amount);
  }
```

Impact
As such, after this seed is minted, totalSupply() can never return to 0, given the dead address has no private key and can never call withdraw/redeem, its shares can never be burned. Therefore totalSupply() >= seedShares > 0 holds forever, the guard in sweep always reverts, and sweep becomes permanently unreachable dead code under the intended deployment.

Recommendation	Consider resolving the conflict between the seed deposit and the sweep guard. Either change the condition under which sweep is allowed to run so it does not depend on totalSupply() == 0, for example by accounting for the fixed seed shares held by the dead address, or remove sweep if the seed based inflation protection makes it unusable.
Status	Fixed, ListaDao has added a code comment indicating the following: /// @dev NOTICE: under the intended dead-seed deployment supply never returns to 0, so this is effectively /// unreachable. Retained intentionally as a safety net for non-seeded/edge deployments; left as-is.

[I01] Redundant write to lastTotalAssets in depositBNB	
Contract	CollateralYieldVault.sol
Severity	Informational

Description
The CollateralYieldVault.depositBNB() function assigns lastTotalAssets = newTotalAssets and then overwrites lastTotalAssets again later in the same function without any read of the variable occurring in between. The first assignment therefore has no effect on the final state or on any intermediate logic. This differs from deposit/mint, where the value written by _deposit is actually read before being updated

Description

JavaScript

```
 function depositBNB(address receiver) external payable nonReentrant
whenNotPaused returns (uint256 shares) {
    _checkWhiteList(receiver);
    if (msg.value == 0) revert ZeroAmount();

    uint256 newTotalAssets = _accrueFee();
    lastTotalAssets = newTotalAssets;
```

```
    uint256 balBefore = IERC20(SLIS_BNB).balanceOf(address(this));
    STAKE_MANAGER.deposit{ value: msg.value }();
    uint256 assets = IERC20(SLIS_BNB).balanceOf(address(this)) -
balBefore;

    shares = _convertToSharesWithTotals(assets, totalSupply(),
newTotalAssets, Math.Rounding.Floor);
    if (shares == 0) revert ZeroAmount();

    _mint(receiver, shares);
    emit Deposit(_msgSender(), receiver, assets, shares);

    PROVIDER.supplyCollateral(marketParams, assets, address(this), "");
    _updateLastTotalAssets(newTotalAssets + assets);
  }
```

Impact

The redundant write does not introduce a security risk, but it wastes gas and reduces code clarity.

Recommendation	Consider removing the first assignment to lastTotalAssets in depositBNB() and keep only the later write that determines the final value. This saves gas and makes the data flow easier to follow.
Status	Fixed, the unnecessary update is removed and replaced with a code comment highlighting the purpose of removal.

[I02] Fee recipient can receive shares while not whitelisted in _accrueFee	
Contract	CollateralYieldVault.sol
Severity	Informational

Description

Description

The CollateralYieldVault._accrueFee() function mints fee shares to feeRecipient without checking whether that address is whitelisted. Minting is exempt from the whitelist enforcement in _update, so the shares are created regardless of the whitelist status of the recipient.

JavaScript

```
function _accrueFee() internal returns (uint256 newTotalAssets) {
    uint256 feeShares;
    (feeShares, newTotalAssets) = _accruedFeeShares();
    if (feeShares != 0) _mint(feeRecipient, feeShares);
    emit AccrueInterest(newTotalAssets, feeShares);
  }
```

	Impact As a result, a feeRecipient that is not on the whitelist can still redeem the accrued shares, but is unable to transfer them. This creates an inconsistency between how fee shares are minted and how the whitelist is meant to restrict share movement.
Recommendation	Consider that when a whitelist is active, require that isWhiteList(feeRecipient) returns true before minting fee shares. This keeps the feeRecipient consistent with the whitelist policy applied to all other shareholders, and allows admin to potentially blacklist a feeRecipient.
Status	Acknowledged, no fix implemented, it is presumed that the feeRecipient will not be removed from the whitelist and will be a ListaDao trusted address.

[I03] MPC wallet capacity limits in SlisBNBxMinter can cause deposit reverts	
Contract	CollateralYieldVault.sol
Severity	Informational
Description	**Description** When users call deposit, depositBNB, or mint on CollateralYieldVault, the flow eventually reaches the _mintToMPCs() function in SlisBNBxMinter. Each MPC wallet has a hard cap on the number of LP tokens it can hold. The _mintToMPCs() function loops through all configured wallets and distributes the newly reserved LP tokens among them. **Impact** If the combined remaining capacity across all wallets is not enough to absorb the new reserved LP tokens, the function reverts. This causes the user's deposit or mint transaction to fail, blocking deposits whenever total MPC wallet capacity is exhausted.
Recommendation	Ensure that the total MPC wallet capacity is monitored and expanded ahead of demand, for example by adding new wallets or raising caps before existing capacity runs out
Status	Acknowledged, no fix implemented, MPC wallet capacity is presumed to have sufficiently high capacity set.

[I04] Deposits before setDelegateTarget mis-route slisBNBx to the vault	
Contract	CollateralYieldVault.sol
Severity	Informational
Description	**Description** If a deposit/mint or depositBNB occurs while delegation[vault] is still address(0), the mint path in SlisBNBxMinter._rebalanceUserLp reads the delegation holder as

	address(0), then defaults the holder to the account itself and silently pins delegation[account] to the CollateralYieldVault.sol.
	
	JavaScript
	`// account as the default delegatee if holder is not set`
	`if (holder == address(0)) {`
	`holder = account;`
	`delegation[account] = holder;`
	`}`
	
	Impact
	As a result,
	• The launchpool slisBNBx is minted to the vault rather than to the intended launchpool MPC. The intended MPC earns nothing until a later setDelegateTarget call migrates the position. That migration does function correctly, since the old delegatee is the vault, the vault's balance is burned, and the tokens are minted to the target, but only while the vault still holds that slisBNBx.
	• Another risk arises during the window in which the slisBNBx sits in the vault. The MANAGER can call vault.emergencyWithdraw(amount, slisBNBx) and pull those tokens out. Once that happens, balanceOf(vault) becomes less than userTotalBalance[vault], so the later delegate switches under burns and triggers an accounting desync.

Recommendation	Consider setting the delegation target through setDelegateTarget before any deposits are allowed. This can be performed either • Directly via the initialize function • Indirectly by including a non-zero address check within deposit/mint/depositBNB functions for the delegateTarget address
Status	Fixed, the deploy_collateral_yield_vault script already atomically calls the setDelegateTarget, so this attack vector is not applciable.

[I05] setDelegateTarget lacks guards for same target and fee MPC wallet collisions	
Contract	CollateralYieldVault.sol
Severity	Informational
Description	**Description** The CollateralYieldVault.setDelegateTarget() function does not guard against two misconfiguration cases.

1. First, when the target is switched to the same address that is already set, the underlying minter performs no operation since it returns early, but the vault still updates delegateTarget and emits an event. This produces a misleading state change and event for an action that had no effect.

```javascript
 function _delegateAllTo(address account, address newDelegatee) internal
{
    if (newDelegatee == delegation[account]) {
      return;
    }
```

2. Second, if the delegatee is set to an address equal to a fee MPC wallet, the fee ledger can be corrupted, because the burn used during the switch operates on shared balances and does not distinguish fee account tokens from delegated ones.

The impact is limited to cosmetic noise in the first case and an accounting corruption that requires a misconfiguration in the second case.

Recommendation	Consider adding a check in setDelegateTarget() that reverts or returns early when the new target equals the current delegateTarget. Also ensure that the target delegate never equals a fee MPC wallet.
Status	Fixed, now includes an input check that ensures that the target delegate to set is different from the current delegateTarget. It is also presumed that the MANAGER role will not update a target delegate to be equal to the feeRecipient address.

[I06] Dead seed shares capture a permanently locked pro-rata portion of vault yield	
Contract	CollateralYieldVault.sol
Severity	Informational
Description	**Description** The deployment script seeds the CollateralYieldVault with a fixed 0.02 BNB first deposit whose shares are minted to the burn address 0x...dEaD. These seed shares are ordinary ERC4626 shares and are priced identically to every other holder's shares through convertToAssets(), which derives value from totalAssets(). As a result, the dead address participates in all share price increases and captures a pro-rata slice of every reward injected through increaseVaultAssets(), and it is correspondingly diluted by performance fee shares minted to feeRecipient when fee > 0.

JavaScript

```
   // 3. dead seed deposit: keeps totalSupply > 0 forever (whitelist
still empty == open here).
    uint256 seedShares = vault.depositBNB{ value: SEED_BNB }(DEAD);
    console.log("Seed shares minted to dead address:", seedShares);
    require(vault.totalSupply() == seedShares && seedShares > 0, "seed
failed");
```

Impact

At healthy TVL, this dead shares impact on yield dilution is not significant. It is about 1.96% of yield at 1 BNB TVL, about 0.2% at 10 BNB, about 0.02% at 100 BNB, and negligible beyond that, with the absolute locked value limited to the share of appreciation from the initial 0.02 BNB seed

Recommendation	Consider not harvesting rewards until the vault holds meaningful user TVL relative to the 0.02 BNB seed, so that the pro-rata fraction captured by the dead address stays negligible. Additionally, document that the seed's appreciating value is intentionally unrecoverable,
Status	Acknowledged, this is presumed to be a design choice to prevent inflation attacks.

[I07] Harvested event in harvest mis-reports claimed epochs and BNB	
Contract	RewardHarvester.sol
Severity	Informational

Description

The RewardHarvester.harvest() function emits Harvested(claims.length, bnbBal). The first value, claims.length, includes epochs that were skipped because they had already been claimed externally. Off chain monitoring that interprets these values as the number of epochs claimed in this call will be incorrect.

Description

JavaScript

```
 function harvest(ClaimParams[] calldata claims, uint256 minBNBOut)
external onlyRole(BOT) nonReentrant {
    for (uint256 i; i < claims.length; ++i) {
      // The Merkle tree's `_account` is configured off-chain to this
RewardHarvester, so the distributor pays the
      // launchpool BNB to this contract — receiving it on behalf of the
vault — and we then compound it in.
      // Skip already-claimed epochs to keep the batch idempotent.
      if (DISTRIBUTOR.claimed(claims[i].epochId, address(this)))
continue;
      DISTRIBUTOR.claim(claims[i].epochId, address(this),
claims[i].amount, claims[i].proof);
    }
```

```
    uint256 bnbBal = address(this).balance;
    if (bnbBal == 0) revert NothingToCompound();
    if (bnbBal < minBNBOut) revert InsufficientReward();

    VAULT.increaseVaultAssets{ value: bnbBal }();

    emit Harvested(claims.length, bnbBal);
  }
```

Impact
The impact is limited to cosmetic inaccuracies in the event emitted during harvesting

Recommendation	Consider tracking the count of epochs actually claimed during the call and the BNB amount actually claimed during the call, and pass those into the Harvested event
Status	Fixed, now initializes a claimCount and only increments and emits it in the Harvested event if the claim has not been skipped.

[I08] Permissionlessly claimed BNB stays idle until the next BOT harvest

Contract	RewardHarvester.sol
Severity	Informational

Description
In ClisBNBLaunchPoolDistributor.claim, direct permissionless claims are possible as long as a proof is valid. Such direct claims move BNB into the harvester, but only during actual harvest() calls, which is BOT gated, or rescue, which is MANAGER gated, can move that BNB out.

Description

JavaScript

```
 function harvest(ClaimParams[] calldata claims, uint256 minBNBOut)
external onlyRole(BOT) nonReentrant {
    for (uint256 i; i < claims.length; ++i) {
      // The Merkle tree's `_account` is configured off-chain to this
RewardHarvester, so the distributor pays the
      // launchpool BNB to this contract — receiving it on behalf of the
vault — and we then compound it in.
      // Skip already-claimed epochs to keep the batch idempotent.
      if (DISTRIBUTOR.claimed(claims[i].epochId, address(this)))
continue;
      DISTRIBUTOR.claim(claims[i].epochId, address(this),
claims[i].amount, claims[i].proof);
    }

    uint256 bnbBal = address(this).balance;
```

```
    if (bnbBal == 0) revert NothingToCompound();
    if (bnbBal < minBNBOut) revert InsufficientReward();

    VAULT.increaseVaultAssets{ value: bnbBal }();

    emit Harvested(claims.length, bnbBal);
  }
```

Impact

As a result,

- Externally claimed BNB sits in the harvester without being injected into the vault and does not earn vault yield until the BOT next runs. There is no loss of funds, only a slight latency, and a third party cannot force the BOT to act.
- The MANAGER role may have a window to rescue funds that were intended for the CollateralYieldVault.sol instead of actually rescuing stranded tokens.

Recommendation	Consider • Ensuring that the BOT role harvests more frequently, • Alternatively, consider not allowing rescue of native BNB and instead have all native BNB claimed to be delegated to the vault within the rescue function
Status	Acknowledged, it is presumed that the BOT role will harvest and set intervals appropriately.

[I09] Missing zero share check in deposit() and missing zero asset check in mint()	
Contract	CollateralYieldVault.sol
Severity	Informational
Description	**Description** The deposit() function derives the number of shares from the supplied assets using floor rounding and then calls the internal _deposit function without verifying that the resulting shares value is greater than 0. The protection against the zero case on this path is currently delegated to a revert inside SlisBNBProvider.supplyCollateral as the provider enforces require(assets > 0). Additionally, the mint() function lets the caller specify the share amount directly, so it cannot silently mint 0 shares for a paying caller, and a request of 0 shares reverts through the same provider check eventually.

	The depositBNB() function already contains an explicit guard, if (shares == 0) revert ZeroAmount(). The absence of the equivalent guard in deposit() and mint() is therefore inconsistent.
	
	JavaScript
	`function deposit(`
	`uint256 assets,`
	`address receiver`
	`) public override nonReentrant whenNotPaused returns (uint256 shares)`
	`{`
	`_checkWhiteList(receiver);`
	`uint256 newTotalAssets = _accrueFee();`
	`lastTotalAssets = newTotalAssets;`
	`shares = _convertToSharesWithTotals(assets, totalSupply(), newTotalAssets, Math.Rounding.Floor);`
	`_deposit(_msgSender(), receiver, assets, shares);`
	`}`
	
	`/// @inheritdoc IERC4626`
	`function mint(uint256 shares, address receiver) public override nonReentrant whenNotPaused returns (uint256 assets) {`
	`_checkWhiteList(receiver);`
	`uint256 newTotalAssets = _accrueFee();`
	`lastTotalAssets = newTotalAssets;`
	`assets = _convertToAssetsWithTotals(shares, totalSupply(), newTotalAssets, Math.Rounding.Ceil);`
	`_deposit(_msgSender(), receiver, assets, shares);`
	`}`

Recommendation	Consider including an explicit if (shares == 0) revert ZeroAmount(); check to deposit() and an explicit if (assets == 0) revert ZeroAmount(); check to mint()
Status	Fixed as recommended, now includes a zero share check and zero assets check after conversion for deposit and mint respectively.